Advanced Algorithms: Convex Hull

Graham scan vs Jarvis march vs Chan's output-sensitive algorithm. When does output sensitivity pay off?

1. The Problem and Algorithms

1.1 Problem Definition

The convex hull problem is a computational geometry problem that finds the smallest convex polygon containing a given set of points in a two-dimensional Cartesian plane. Given a set of **n** points, the goal is to identify the vertices that form the outer boundary (hull) of the polygon, excluding points that are strictly contained within.

The problem can be approached using several algorithms that differ in their construction methods, computational complexity and input sensitivity. Investigate three algorithms, Graham Scan, Jarvis March and Chan's algorithm. Although they all aim to produce the same convex hull, they employ different strategies for identifying the outermost points and constructing the resulting hull.

1.2 Algorithm Description

Graham Scan

Graham Scan constructs the convex hull by sorting the input points according to their polar angle relative to a selected pivot point. The pivot point is determined by identifying the point with lowest vertical coordinate, using the point with the lowest horizontal coordinate as a tie-breaker if multiple points share the same y-coordinate. This sorting establishes the order in which the points are processed to begin constructing the hull.

The algorithm uses a stack to store the vertices currently considered to be part of the hull. As the algorithm examines each point, it evaluates the orientation of the most recent three points using a cross-product calculation. If the points form a clockwise turn, the most recently added point is discarded from the stack, as retaining it would introduce an inward bend in the hull. The process continues until all points have been examined, leaving the vertices of the hull in the stack.

The defining characteristic of this algorithm is that it establishes an ordering of input points prior to hull construction, efficiently eliminating points that are inconsequential to the convex boundary.

Jarvis March

Jarvis March, also known as Gift Wrapping, constructs the convex hull by identifying its boundary vertices sequentially. Starting from an extreme point, typically the leftmost point, the algorithm searches through the remaining input points to determine which point forms the next edge of the convex hull.

For each current hull vertex, the algorithm examines the other input points and uses orientation tests to identify the next boundary vertex. Once this point is selected, it becomes the starting point for the next iteration. The process continues until the algorithm returns to its original starting vertex, completing the convex hull.

Unlike Graham Scan, Jarvis March does not require sorting the entire input beforehand. Instead, it repeatedly examines the input points to identify each successive hull vertex. Its defining characteristic is that its performance depends on both the input size and the number of resulting hull vertices.

Chan's Algorithm

Chan's Algorithm is an output-sensitive convex hull algorithm that combines concepts from Graham Scan and Jarvis March to achieve improved performance across different input distributions.

The algorithm partitions the input points into smaller subsets and independently computes the convex hull of each subset using a sorting-based approach. It then applies a modified version of Jarvis March, using tangent searches over the subset hulls to determine the candidate vertices for the global convex hull.

It also employs an iterative estimation strategy to determine an upper bound on the number of hull vertices. If the algorithm cannot complete the hull within the estimated number of steps, it increases the estimate and repeats the process.

Its defining characteristic is its ability to adapt its computational effort to the size of the final hull rather than relying solely on the total number of input points.

Algorithm Comparison

Algorithm	Approach	Output-sensitive?
Graham Scan	Polar-angle sorting and stack	No
Jarvis March	Repeatedly wrap boundary	Yes

Chan's Algorithm	Partitioning and gift wrapping	Yes

2. Theoretical Comparison and Predictions

2.1 Asymptotic Complexity

The three algorithms all differ in their asymptotic running times and their sensitivity to the number of vertices forming the final convex hull.

Algorithm	Time-Complexity	Auxiliary Space
Graham Scan	O(nlogn)	O(n)
Jarvis March	O(nh)	O(h)
Chan's Algorithm	O(nlogh)	O(n)

In the above table, '**n**' represents the total number of input points, while '**h**' represents the number of points on the convex hull.

Graham Scan: Its asymptotic runtime is independent of the number of hull vertices, as the initial sorting dominates the algorithm's complexity. Regardless of the input distribution, sorted points are processed in linear time during hull construction.

Jarvis March: Its complexity makes it especially sensitive to the number of hull vertices. For example, if *h* is small relative to n, the runtime approaches linear growth, however if a large proportion of points belong to the hull, it approaches quadratic growth.

Chan's Algorithm: This algorithm's complexity combines the benefits of output-sensitivity with a better asymptotic bound than Jarvis March. It can successfully exploit small convex hulls while avoiding the potential worst-case behaviour of Jarvis March, with its advantage being most apparent when 'h' is substantially smaller than 'n'.

2.2 Constant-Factor and Memory Characteristics

While asymptotic complexity can describe how the algorithm scales with increasing input sizes, its performance is also impacted by operations and memory requirements involved in implementation.

Graham Scan: The sorting step is the most expensive operation, requiring comparisons and memory for storing the sorted input vertices. However after sorting, the stack based construction is relatively simple and performs linear total work.

Jarvis March: While it avoids the overhead from sorting input points, it repeatedly scans the entire input and needs to perform orientation calculations to identify successive hull vertices. The memory requirements for this algorithm is relatively low, as it primarily needs to store the convex hull.

Chan's Algorithm: The overhead is derived from partitioning the input, constructing local hulls, performing tangent searches and potentially repeating these processes with larger group sizes. It also requires additional memory to store local hulls and intermediate results.

2.3 Input Characteristics

(n) - Input size: The total number of points provided to the algorithm. Incrementing this value would increase the amount of data needing to be processed.

(h) - Hull size: The number of points belonging to the final convex hull. This has a direct influence on performance for Jarvis March and Chan's Algorithm, with both algorithms being output-sensitive.

(m) - Group size: This represents the maximum number of input points assigned to individual groups during an attempt. The algorithm uses doubly exponentially increasing values to increase this variable if it cannot complete the construction of the global hull with the current limit. (Exclusive to Chan's Algorithm).

Point Distribution: The spatial arrangement of points can influence the size of the convex hull. For example, a small cluster of points would produce a hull with less vertices whereas points distributed in a circle can produce a hull that contains a large proportion of input points.

2.4 Predictions

Input Scenario	Predicted Algorithm	Reasoning

Small datasets	Graham Scan	Smaller implementation overhead compared to other algorithms.
Circular distribution	Graham Scan	Likely to produce a hull with more vertices, diminishing Chan's asymptotic advantage as h approaches n.
Large datasets	Chan's Algorithm	Since 'h' is always less than or equal to 'n' it can never be worse than Graham Scan.
Collinear distribution	Graham Scan	Sorting based approach should offer better efficiency compared to the wrapping approach of the other two algorithms.
Uniform random distribution	Chan's Algorithm	Hull would be small relative to its input size.
Clustered distribution	Chan's Algorithm	Ultimately depends on the resulting h value, but Chan's should outperform the other algorithms.

3. Experimental Methodology

3.1 Metrics

The experiments use execution time as the primary comparison measurement of each convex hull algorithm, allowing performance to be compared across different input sizes and point distributions. Input size and the number of vertices in the computed hull are also recorded, as well as hull size which is useful for interpreting the performance of output sensitive algorithms.

Metric	**Measures**	**Relevance**
Execution time (ns or ms)	Time taken to generate the convex hull	Primary performance comparison
Input size (n)	Total number of input points	Measures scaling as datasets grow
Hull size (h)	Number of vertices in the generated hull	Investigates output-sensitive algorithms

3.2 Input Families and Generation

The program uses four input distributions to investigate how different point arrangements may influence algorithm performance. The distributions represent a range of both convex hull sizes and geometric structures.

Uniform Random: Input points are distributed randomly across the coordinate region. It provides a general baseline for evaluating performance under irregular distributions.

Circular: Points are placed at random angles around the centre of the coordinate region, with radii ranging from 350 to 500. This generates a ring rather than points all lying in one circle, meaning the hull may contain substantially fewer vertices than the dataset.

Collinear: Points are generated using the formula ($y = 0.5x + 25$). Floating point calculations can produce slight deviations from exact collinearity.

Clustered: Points are assigned in turn to three fixed centres, with normally distributed coordinate offsets. The generated clusters can grow beyond the 0 - 1000 range.

The API accepts dataset sizes from 0 to 10,000 points. The benchmark accepts a list of sizes and runs each algorithm on the generated dataset for each size. The sizes are supplied in the request and the program does not define a fixed default.

3.3 Benchmarking Setup

The experiments were conducted using implementations of Graham Scan, Jarvis March and Chan's Algorithm, written in Go.

The benchmark evaluates the three algorithms against the established input distributions and their set dataset sizes. Algorithms can all be run sequentially on the same dataset, allowing execution times and hull sizes to be evaluated under equivalent conditions.

Category	Specification
Programming Language	Go
Version	1.27.1
Operating System	Arch Linux (Linux kernel 7.2.6)
CPU	AMD Ryzen 7 7800x3D
RAM Capacity	64 GB

Input generation and hull construction are treated as separate operations so the execution time only reflects the algorithm's computational work and not the time required to generate the dataset.

3.4 Repetitions and Variance

The benchmarking suite permits execution across 1 to 100 trials per request. Rather than computing summary metrics automatically, it yields raw timing values per trial. Consequently, when reporting median values across three iterations, these figures are derived directly from the recorded dataset.

Subsequent iterations reuse identical input configurations for a given dataset size and random seed. Because execution order remains static, hardware caching, system overhead, and warm-up dynamics can affect latency metrics. Introducing randomized trial ordering alongside preliminary warm-up passes would better isolate these underlying effects.

3.5 Fairness and Reproducibility

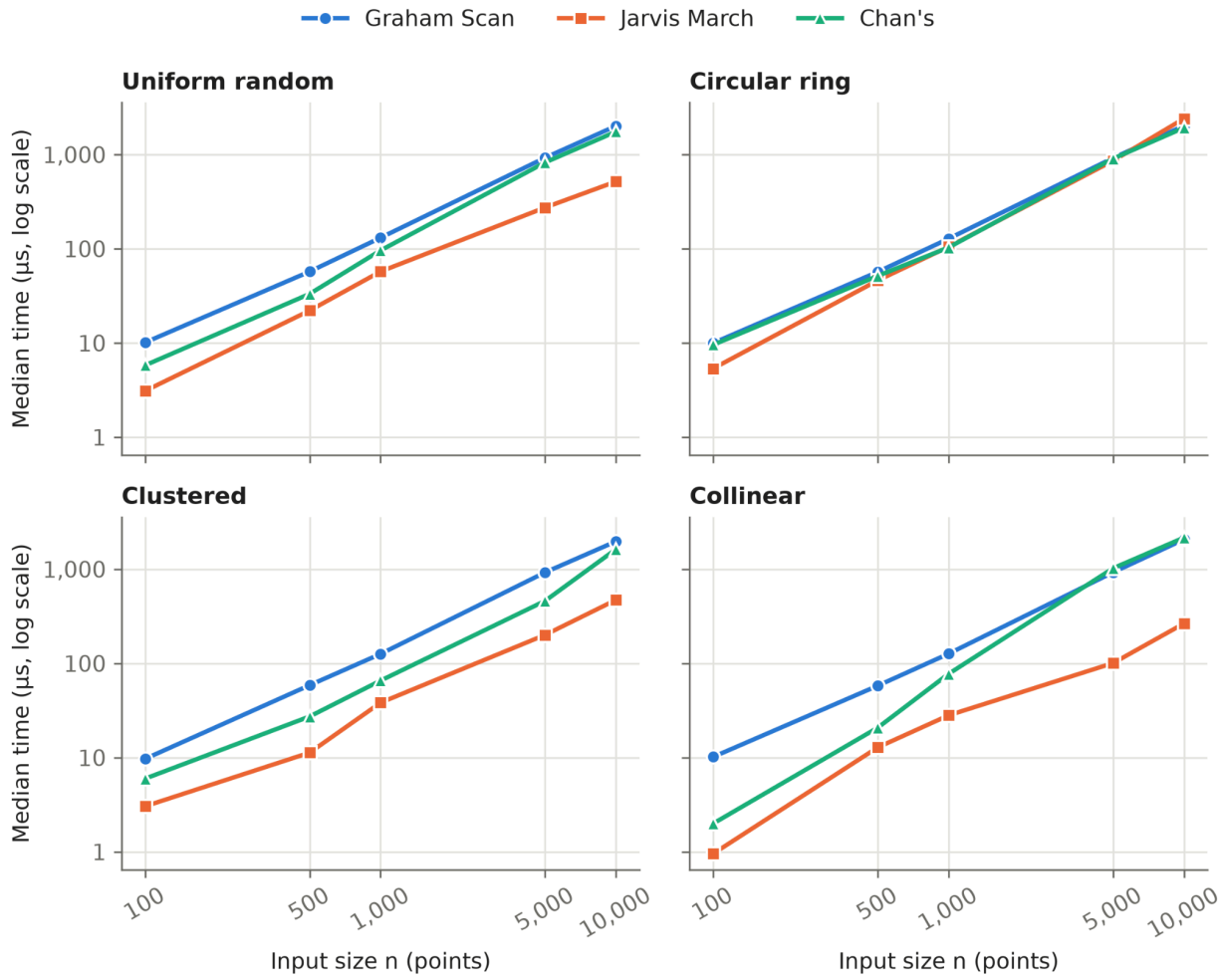
To support fair comparisons, each algorithm received the same generated input points, and input generation was excluded from the timed computation. Datasets were generated from a specified distribution, size, and seed, allowing the same inputs to be reproduced.

The algorithms were measured sequentially rather than concurrently. Their execution order was fixed, so the results may still be affected by factors such as runtime warm-up, caching, or system activity. Randomizing execution order, adding warm-up runs, and increasing the number of trials would help assess these effects in future experiments.

4. Results and Analysis

Median of three trials, in microseconds (μs).
H is the number of hull vertices.
The fastest time in each is in bold.

4.1 Results by distribution



Uniform Random

n	h	Graham Scan	Jarvis March	Chan's
100	11	10.1	**3.1**	5.8
500	16	57.4	**21.5**	36.6
1,000	21	131.0	**55.5**	121.7
5,000	20	926.5	**264.2**	863.6
10,000	19	2,015	**499.3**	1,734

Circular Ring

n	h	Graham Scan	Jarvis March	Chan's
100	20	10.0	**5.5**	9.8

500	34	56.9	**45.7**	57.8
1,000	39	128.0	**106.0**	133.7
5,000	63	920.4	**822.4**	922.0
10,000	88	2,027	2,267	**1,929**

Clustered

n	h	Graham Scan	Jarvis March	Chan's
100	11	9.7	**3.0**	6.1
500	8	58.7	**11.0**	29.1
1,000	14	126.0	**37.7**	76.9
5,000	14	924.9	**186.9**	457.0
10,000	17	1,974	**446.9**	1,635

Collinear (h differs slightly between algorithms, see Section 5)

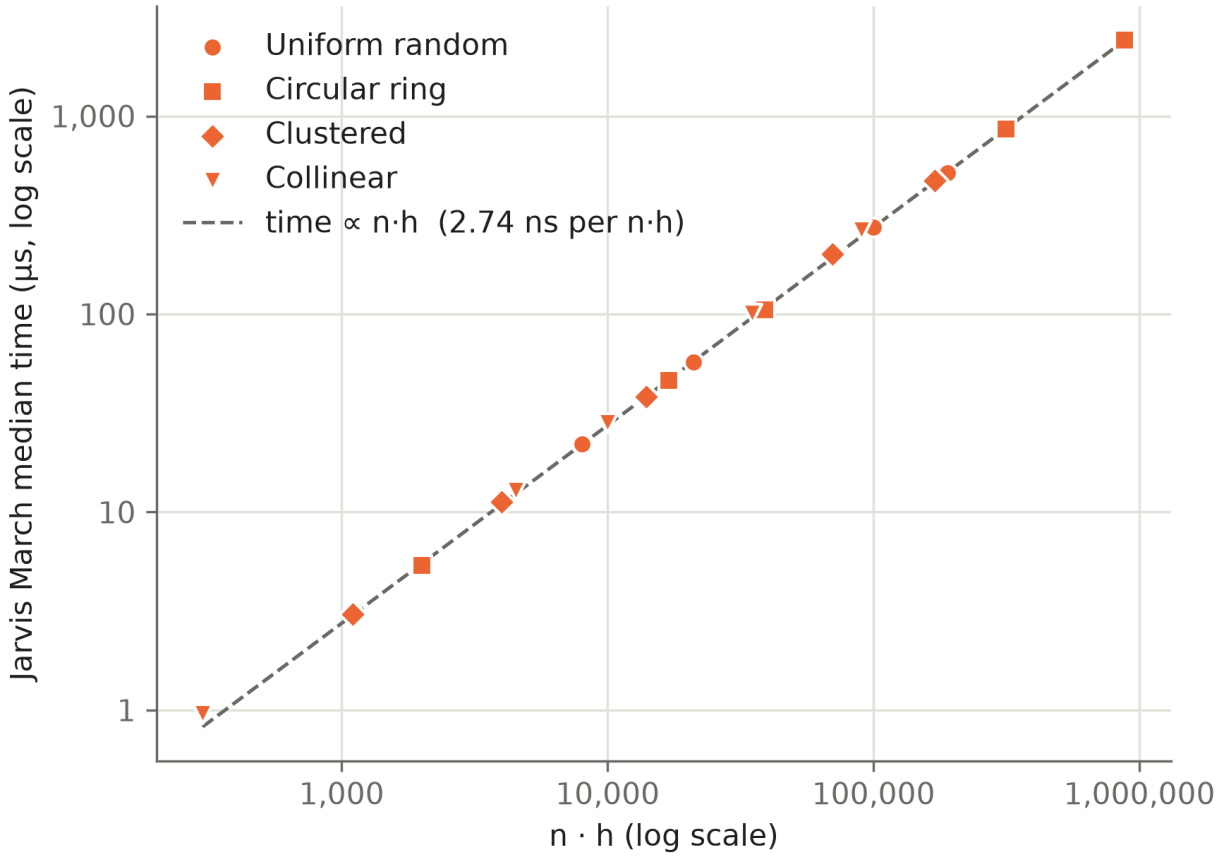
n	h	Graham Scan	Jarvis March	Chan's
100	3-4	10.2	**1.0**	2.1
500	6-9	57.9	**13.3**	22.0
1,000	5-10	126.6	**29.9**	85.3
5,000	7-13	926.5	**114.1**	1,060
10,000	9-17	2,070	**300.4**	2,203

4.2 Analysis

Graham Scan was the slowest algorithm in 18 of 20 configurations; Chan's was slower on collinear data at n = 5,000 and 10,000. At n = 10,000 it took 1.97–2.07 ms on every distribution, almost exactly the same whether the hull had 9 or 88 vertices. This confirms it is not output-sensitive. From n = 100 to n = 10,000 its runtime grew about 200×, which is what O(n log n) predicts.

An earlier version computed polar angles with atan2, which profiling showed took 59% of the runtime. Replacing it with a cross-product comparison made Graham Scan about 2.3× faster (4.6 → 2.0 ms at n = 10,000). Profiling the new version shows about 58% of its time goes to sorting and about 34% to removing duplicate points with a hash map. These constant factors explain why it still loses on small hulls.

Jarvis March was fastest in 18 of 20 configurations. Its runtime closely follows n·h, as theory predicts. At n = 10,000:
- Circular has 4.6× the n·h of uniform (88 vs 19 hull vertices), and took 4.7× as long (2,411 vs 518 µs).
- Colinear has the smallest hulls and gave Jarvis its best results.



Because every dataset here has a small hull (at most 88 vertices), Jarvis's simple inner loop of one orientation test per point per hull vertex beats both sorting-based algorithms.

It won 2 configurations (circular n = 1,000 and n = 10,000), placed second in 16, and was slowest on collinear n = 5,000 and 10,000. It beat Graham Scan in 18 of 20 configurations.

4.3 Controlled hull size

The standard distributions never produce large hulls, so a supplementary experiment controlled h directly. It used 100,000 points: h vertices of a regular polygon plus random points inside it. Times are medians of three trials, in milliseconds (ms).

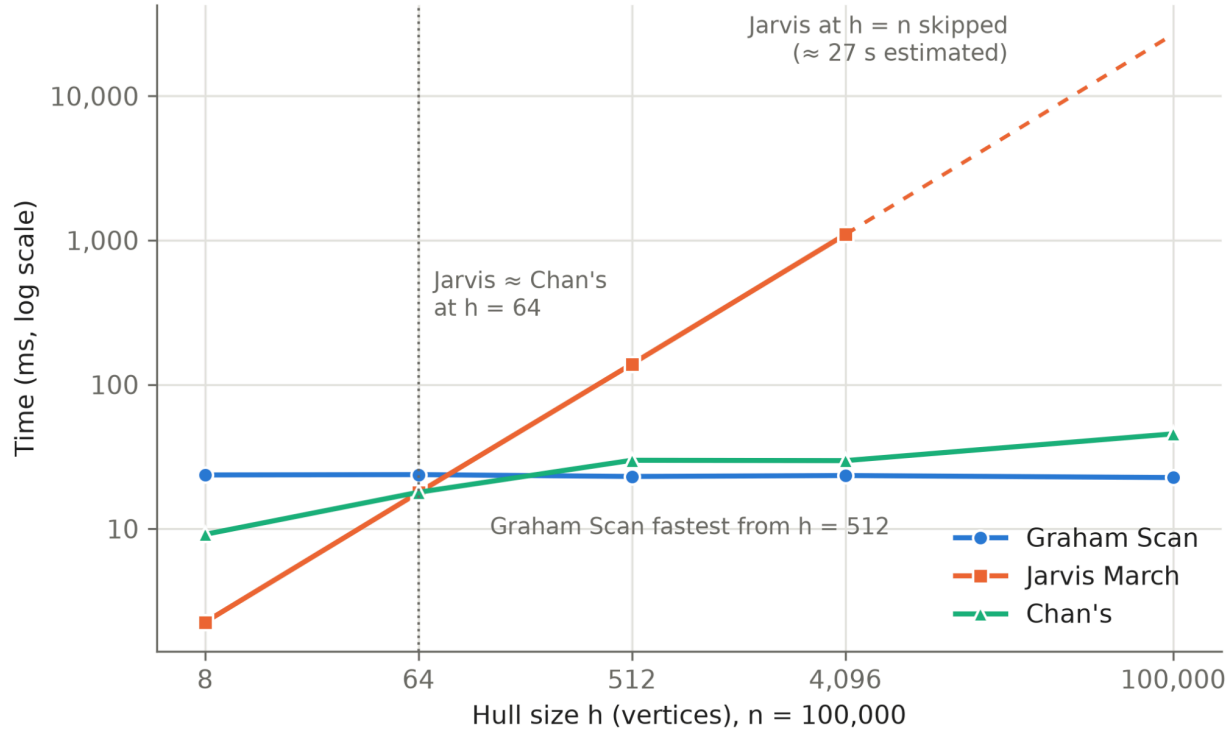
h (n = 100,000)	Graham Scan (ms)	Jarvis March (ms)	Chan's (ms)

8	23.6	**2.3**	9.1
64	23.7	**17.8**	17.9
512	23.0	138.8	**29.9**
4,096	23.4	1,098	**29.7**
100,000	22.7	Skipped (≈ 27 s estimated)	**45.5**

These results show the theory clearly:

- Graham Scan's time is flat across h (about 23 ms).
- Jarvis's time grows in proportion to h (8x more vertices took about 7.7x longer).
- Chan's time grows only with log h (512x more vertices took about 3.3x longer).-

Jarvis and Chan's are level at h = 64 (17.8 vs 17.9 ms). From h = 512 upward Graham Scan is fastest, so Chan's is never the fastest at n = 100,000: its better asymptotic bound doesn't make up for its higher constant factor at this size. With every point on the hull at n = 10,000, Jarvis took 289.5 ms, against 2.0 ms for Graham Scan and 3.0 ms for Chan's.



4.4 Predictions vs. results

Scenario	Predicted	Observed	Correct?

Small data sets	Graham Scan	Jarvis March. Graham was slowest, 2–10× behind Jarvis at n = 100	x
Circular	Graham Scan	Jarvis at n = 100, 500 and 5,000. Chan's at 1,000 and 10,000, both by small margins. Small hulls (h ≤ 88)	x
Large datasets	Chan's	Jarvis at n = 10,000 except circular (Chan's). In the controlled test, Graham Scan wins once h >= 512 and Chan's never wins	Partly
Collinear	Graham Scan	Jarvis March, Graham beat Chan's at n = 5,000 and 10,000	x
Uniform random	Chan's	Jarvis March with Chan's second	x
Clustered	Chan's	Jarvis March, with Chan's second	x

5. Surprising Facts

5.1 Best Performing Algorithm

The experimental results revealed several differences between the theoretical expectations and actual algorithm performance, exposing limitations in dataset generation and displaying how implementation decisions influenced benchmark results.

The most unexpected finding was that Jarvis March, despite having the worst-case complexity of (O(n^2)), achieved the fastest execution time in 18 of 20 standard benchmark configurations.

This is largely explained by the relatively small hull sizes produced by the datasets, ranging from 3 to 88 vertices. Given that Jarvis March is one of the two output sensitive algorithms, its performance remained competitive given that the value of 'h' was small relative to the total input size.

5.2 Graham Scan Runtime

Graham Scan's runtime depended heavily on one implementation choice. Computing polar angles with atan2 took 59% of its runtime. Replacing it with a cross-product comparison made it about 2.3× faster (57 → 23 ms at n = 100,000). After the change, Graham Scan beats Chan's whenever the hull is large: from h = 512 at n = 100,000, and with every point on the hull at n =

10,000 (2.0 vs 3.0 ms). Both are O(n log n) in that case, so the difference comes down to constant factors: Chan's repeated grouping, sorting and tangent searches over several rounds..

5.3 Circular Dataset Hulls

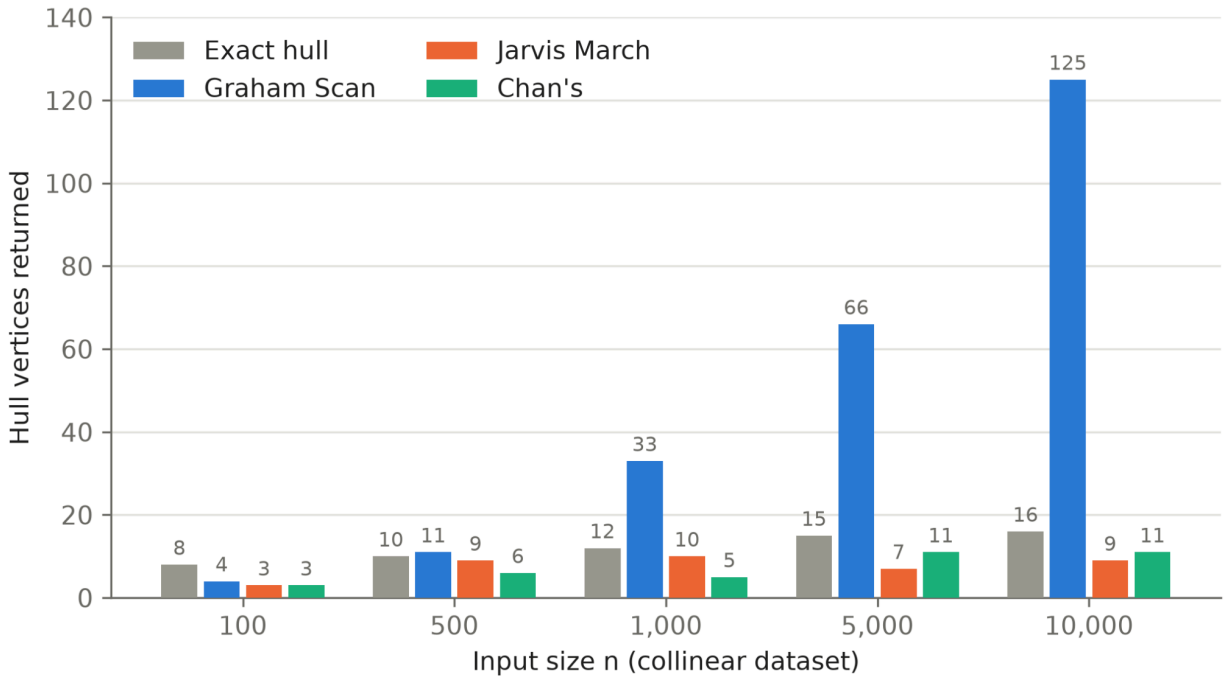
The circular dataset did not produce the expected hull size, as the generated points were assigned a random radius between 350 and 500. This resulted in an annulus shaped distribution, rather than every input point being placed directly on a circle's circumference. Thus, many of the generated points were located inside the convex hull rather than being included in the global boundary.

The resulting hulls contained between 20 and 88 vertices, meaning the dataset did not effectively represent the expected worst-case scenario for output-sensitive algorithms, Jarvis March in particular.
Identifying this distinction acknowledges that a true circular boundary distribution was not tested, as a true circular distribution with distinct points positioned on the circumference would produce a hull that contains approximately every input point.

5.4 Collinear Dataset Deviations

The collinear dataset also behaved differently than intended. Although the generator aimed to produce points along a straight line, floating-point rounding introduced deviations in their coordinates, resulting in generated points being near-collinear rather than perfectly collinear. Floating-point rounding moves points up to one rounding step off the line $y = 0.5x + 25$, so the true hull is a very thin polygon (16 vertices at n = 10,000, verified with exact rational arithmetic).

At (n=5000), Graham Scan produced 66 hull vertices (the exact hull has 15), Jarvis March 7 and Chan's Algorithm 11. At n = 10,000 Graham Scan returned 125 vertices against an exact 16. These differences demonstrate the sensitivity of geometric orientation calculations to floating point precision. None are exactly correct, but in every case no input point lies more than 2.1×10^{-10} outside the returned hull. On exactly collinear integer inputs, and on all non-degenerate datasets, all three algorithms return the exact hull. The discrepancy is therefore a shared numerical-robustness limitation, not an implementation error. This got worse after the cross-product change. When points are almost in a line, rounding makes the comparison give inconsistent answers, so the sort order is unreliable and the scan keeps extra vertices. The atan2 version hid this.



This issue caused Jarvis March to enter an endless loop during development, which was caught by the test suite.

Chan's algorithm experienced additional performance difficulties. The gift-wrapping procedure required four rounds to close the hull with 391 tangent searches at n = 10, 000 needing a slower linear-scan fallback. Consequently, it performed worse on collinear data than on uniform random data at the same input size.

6. Future Work

Implementation Improvements: Alternatively, Andrew's Monotone Chain can be implemented as another sorting-based comparison, helping distinguish the cost of the underlying algorithm from overhead introduced by implementation choices. Duplicate removal now takes about 34% of Graham Scan's time; sorting first and then dropping adjacent duplicates would avoid the hash map.

Numerical Reliability: Robust geometric predicates using exact or adaptive-precision arithmetic to replace the current floating-point orientation calculations. This would help reduce inconsistencies when processing near-collinear points and ensure that all three algorithms produce consistent convex hulls for the same generated input. Robust predicates would also fix Graham's collinear result (125 vs 16).

Benchmarking Methodology: The benchmarking methodology can be improved by introducing warm-up executions, randomising the execution order in which the algorithms are and increasing the number of trials. Reporting confidence intervals would also provide a stronger indication of measurement reliability. Additionally, including memory allocation measurements alongside execution time could be incorporated. The test suite displays differences in memory usage, with Jarvis March making a single 256-byte allocation, whereas Chan's Algorithm allocates approximately 2.4 times the input size. Further investigation could identify how these differences scale on larger datasets.

Cross-Hardware Benchmarking: Repeat experiments on different hardware configurations to observe whether performance differences are consistent across a variety of processors and systems. This would help determine the extent to which results are influenced by hardware-specific characteristics rather than general algorithmic behaviour.

Additional Algorithms: Introduce additional algorithms, such as QuickHull or Kirkpatrick-Seidel to broaden the comparison and whether observed performance trends extend to alternative algorithmic approaches.

7. Individual Contributions

Zeeshan Ahmed
Worked on the group report, establishing the initial report structure and writing up the first three sections, editing later sections to improve depth of explanation.

Ali Abbas
Set up the base repository and implemented the testing suite and all additional features required to run and display the performance of the three algorithms. Ran tests and wrote the results, created diagrams and made presentation slides.

8. AI Use and Reflection

8.1 Individual AI Tool Disclosures

Ali

Claude Code (Anthropic) was used during development and report writing. It:
- implemented Jarvis March and Chan's Algorithm;
- wrote the correctness and efficiency test suites (shared correctness checks, an independent reference algorithm, operation-count and memory tests);
- ran the benchmark and profiling experiments in Section 4, and drafted Sections 3.4-6.

Zeeshan

ChatGPT was used during development and report writing. It assisted in quickly building an understanding of the convex hull problem and the underlying concepts of Graham Scan, Jarvis March and Chan's algorithm It also provided guidance during the implementation of Graham Scan, clarifying the algorithm's geometric operations and potential implementation bugs. Using the assignment report specifications, it helped draft a structure to follow and suggested improvements for sections that were underdeveloped.

8.2 AI Assistance Examples

- During the initial implementation of Chan's Algorithm, ChatGPT provided guidance for developing a version of the algorithm. However, its shortcomings were quickly identified as it omitted key components of the standard algorithm, particularly the tangent finding procedures. This resulted in an implementation that failed to accurately reflect the algorithm's intended approach or its theoretical asymptotic complexity (O(nlogh)).

- There were significant challenges encountered with the benchmarking environment on a Windows operating system. Initially, our tests behaved inconsistently making it difficult to directly compare and validate the results against those obtained on a Linux distribution. This was resolved by revisiting the benchmark tests and applying a timing fix that would allow for more result similarities between operating systems.

- AI provided misleading assistance during the implementation of Graham Scan. Despite explicitly specifying that the algorithm should use the cross product to determine point orientation rather than atan2, the generated implementation still included it for angle calculations. Rather than scrutinise it sufficiently, this was overlooked and created a discrepancy between the original requirements and the resulting code. This was resolved by Ali identifying the mistake and then rewriting the code to rectify it.

8.3 Reflection on AI in Group Work

AI tools were heavily used throughout this project for understanding both the problem and algorithms, also providing considerable assistance in implementing both the algorithms and the benchmarks to test them. Individually, AI was used to clarify unfamiliar concepts and allowed the group to develop a stronger understanding of their respective responsibilities.

Although a lot of our AI use happened independently rather than collaboratively, the ideas and solutions developed using these tools influenced our individual contributions to the task. Overall, it was most valuable when supplementing our understanding by expanding on unfamiliar parts of both the problem and algorithms